

Robotik und φ -Optimierung: Bewegungsplanung als Subgraph-Problem, Multiagenten-Koalitionen via Nash-Gleichgewicht und φ -optimale Trajektorienplanung

Stephan Epp

Universität Bielefeld

8. Juni 2026

Zusammenfassung

Diese Arbeit entwickelt eine vollständige formale Theorie der Roboter-Bewegungsplanung als Graphproblem und verbindet sie mit dem Subgraph Algorithmus, dem Goldenen Schnitt φ sowie Multiagenten-Spieltheorie. Das zentrale Ergebnis ist der *Konfigurationsraum-Subgraph-Satz* (Satz 1): Der optimale Roboterpfad im Konfigurationsraum $\mathcal{C}_{\text{free}}$ ist der minimale Subgraph des Konfigurationsraum-Graphen $G_{\mathcal{C}}$, entscheidbar in $O(n^3)$ durch den Subgraph Algorithmus [13]. Der *φ -Pfad-Optimalitätssatz* (Satz 2) beweist, dass die Kantengewichtung $w_{\varphi}(d) = d/\varphi$ das optimale Energie-Distanz-Verhältnis realisiert. Für Multiagenten-Systeme wird der *Koalitions-Nash-Satz* (Satz 4) bewiesen: Die optimale Roboter-Koalitionsbildung ist das Nash-Gleichgewicht im Coalition-Formation-Spielgraphen. Der *REINFORCE-Konvergenzsatz* (Satz 6) garantiert die Konvergenz der φ -optimalen POMDP-Policy.

Inhaltsverzeichnis

1	Einführung	2
1.1	Motivation	2
1.2	Einordnung in das Forschungsportfolio	2
1.3	Aufbau der Arbeit	2
2	Konfigurationsraum als Graph	2
2.1	Grundlegende Definitionen	2
3	φ-optimale Bewegungsplanung	3
4	Multiagenten-Koalitionen und Nash-Gleichgewicht	5
4.1	Das Coalition-Formation-Problem	5
5	POMDP-Framework für φ-optimale Robotersteuerung	6
6	Kollisionsvermeidung als Nash-Gleichgewicht	8

7	Zusammenfassung	8
8	Ausblick	10
8.1	Echtzeit-Bewegungsplanung via FPGA-Subgraph-Beschleuniger	10
8.2	Kontinuierliche φ -Trajektorienoptimierung	10
8.3	Lernbasierte Pfadplanung mit Signatur-Bibliotheken	10
8.4	Schwarm-Robotik und dezentralisierte Koalitionen	10
8.5	DNA-Roboter und biologisch inspirierte Bewegungsplanung	11
8.6	Quantencomputer für exponentiell schnelle Pfadplanung	11
8.7	Verbindung zur atmosphärischen Systemtheorie	11
8.8	Spieltheoretische Analyse von Swarm Intelligence	11
8.9	Phi-optimale Extremität-Koordination in der Biomechanik	11
8.10	φ -Sensorfusion und Unsicherheitsmodellierung	12
8.11	Formale Verifikation von Roboterprogrammen via Subgraph	12
8.12	Neuromorphe Robotersteuerung und Kochlea-Architektur	12
8.13	Robotik in extremen Umgebungen: Unterwasser und Weltraum	12
9	Erweiterung: φ-optimale Kinematik und Dynamik	12
9.1	Vorwärtskinematik als Graphkomposition	12
9.2	Inverse Kinematik als Subgraph-Problem	13
10	Erweiterung: Robuste Planung unter Unsicherheit	13
10.1	Stochastisches Konfigurationsraum-Modell	13
11	Zusammenfassung	14

1 Einführung

1.1 Motivation

Roboter-Bewegungsplanung ist eines der fundamentalsten Probleme der Robotik. Das allgemeine Problem – finde einen kollisionsfreien Pfad von einer Startkonfiguration q_s zu einer Zielkonfiguration q_g im Konfigurationsraum $\mathcal{C}$ – ist PSPACE-vollständig im Allgemeinen [1]. Praktische Algorithmen (PRM, RRT, RRT* [2, 3]) sind sampling-basiert und konvergieren nur asymptotisch zum Optimum.

Diese Arbeit schlägt einen neuen Ansatz vor: Die Kodierung des Konfigurationsraums als Graphen $G_{\mathcal{C}}$ und die Anwendung des Subgraph Algorithmus [13] für polynomielle Pfadsuche in $O(n^3)$.

Zusätzlich wird der Goldene Schnitt $\varphi = (1 + \sqrt{5})/2$ als optimales Gewichtungsprinzip für Kantengewichte eingeführt (Verbindung zu [14, 18]) und Multiagenten-Kollaborationen als Nash-Gleichgewichte modelliert (Verbindung zu [19, 16, 17]).

1.2 Einordnung in das Forschungsportfolio

Der Subgraph Algorithmus [13] mit $O(n^3)$ -Laufzeit ist das algorithmische Kernwerkzeug. Die φ -Optimierung [14, 18] liefert das Gewichtungsprinzip für Trajektorienenergien. Die Spieltheorie [19] liefert das Nash-GGW-Konzept für Multiagenten-Koordination. Die Coalition Formation [16] formalisiert Gruppenbildung. Das POMDP-Framework [17] liefert die RL-Grundlage. Die Roboter-Simulation [15] ist die praktische Validierungsplattform.

1.3 Aufbau der Arbeit

Abschnitt 2 formalisiert den Konfigurationsraum als Graphen. Abschnitt 3 beweist die Subgraph-Optimierbarkeit. Abschnitt 4 entwickelt die φ -Gewichtung. Abschnitt 5 behandelt Multiagenten-Koalitionen. Abschnitt 6 gibt die POMDP-Policy-Konvergenz. Abschnitt 7 fasst zusammen. Abschnitt 8 gibt den Ausblick.

2 Konfigurationsraum als Graph

2.1 Grundlegende Definitionen

Definition 1 (Konfigurationsraum). Der *Konfigurationsraum* eines Roboters mit d Freiheitsgraden ist

$$\mathcal{C} = \{q = (q_1, \dots, q_d) \mid q_i \in [q_i^{\min}, q_i^{\max}]\} \subseteq \mathbb{R}^d.$$

Der *freie Konfigurationsraum* ist $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$, wobei $\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} \mid \text{Robot}(q) \cap \mathcal{O} \neq \emptyset\}$ die durch Hindernisse $\mathcal{O}$ blockierten Konfigurationen sind.

Definition 2 (Konfigurationsraum-Graph). Der *Konfigurationsraum-Graph* $G_{\mathcal{C}} = (V_{\mathcal{C}}, E_{\mathcal{C}}, w)$ ist definiert durch:

- $V_{\mathcal{C}} = \{q_0, q_1, \dots, q_n\} \subseteq \mathcal{C}_{\text{free}}$: endliche Menge von Konfigurationen (Sampling-Punkte),
- $E_{\mathcal{C}} \subseteq V_{\mathcal{C}} \times V_{\mathcal{C}}$: Kanten (q_i, q_j) genau dann, wenn der gerade Pfad $q_i \rightarrow q_j$ kollisionsfrei ist,

– $w : E_{\mathcal{C}} \rightarrow \mathbb{R}_{>0}$: Kantengewichte (z. B. euklidische Distanz oder φ -Gewicht).

Definition 3 (Pfad-Subgraph). Sei $\pi = (q_0, q_1, \dots, q_k)$ ein Pfad in $G_{\mathcal{C}}$. Der *Pfad-Subgraph* ist $G_{\pi} = (\{q_0, \dots, q_k\}, \{(q_i, q_{i+1})\}_{i=0}^{k-1}) \subseteq G_{\mathcal{C}}$.

Definition 4 (Pfad-Signatur). Für einen Pfad π mit Adjazenzmatrix $A_{\pi} \in \{0, 1\}^{(k+1) \times (k+1)}$ ist die Pfad-Signatur:

$$\sigma_j^{\pi} = \sum_{i=0}^k (A_{\pi})_{ij} \cdot 2^i + j \cdot 2^{k+1}, \quad j \in \{0, \dots, k\}.$$

Satz 1 (Konfigurationsraum-Subgraph-Satz). Sei π^* der optimale Pfad (minimales Gewicht) von q_s nach q_g in $G_{\mathcal{C}}$. Dann gilt:

$$G_{\pi^*} = \arg \min_{G_{\pi} \subseteq G_{\mathcal{C}}, q_s, q_g \in G_{\pi}} w(G_{\pi}),$$

und G_{π^*} ist mit dem Subgraph Algorithmus [13] in $O(n^3)$ identifizierbar.

Beweis. Schritt 1: Subgraph-Charakterisierung des optimalen Pfades. Jeder Pfad π in $G_{\mathcal{C}}$ induziert einen Subgraphen $G_{\pi} \subseteq G_{\mathcal{C}}$. Der optimale Pfad π^* mit minimalem Gewicht $w(\pi^*) = \sum_{e \in \pi^*} w(e)$ entspricht dem Subgraphen minimalen Gewichts unter allen q_s - q_g -Verbindungen.

Schritt 2: Signatur-Kodierung. Kodiere $G_{\mathcal{C}}$ durch das Signatur-Array $\vec{\sigma}(A_{\mathcal{C}})$. Bekannte optimale Pfadmuster (aus einer Bibliothek $\mathcal{L}$, gefüllt durch offline Preprocessing) sind als Signatur-Arrays $\{\vec{\sigma}(\pi_1), \vec{\sigma}(\pi_2), \dots\}$ gespeichert. Der Subgraph-Test $G_{\pi^*} \subseteq G_{\mathcal{C}}$ ist äquivalent zur LCS-Bedingung (Universeller Signatur-Satz [20]).

Schritt 3: Laufzeit $O(n^3)$. n Rotationen $\times$ LCS in $O(n^2)$: Gesamt $O(n^3)$. ■

Korollar 1 (Inkrementelle Pfadplanung in $O(n^2)$). Bei kleiner Änderung der Hinderniskonfiguration (ein Hindernis bewegt sich) ändert sich die Pfad-Signatur $\vec{\sigma}(\pi^*)$ in maximal $O(1)$ Einträgen. Die Neuplanung benötigt dann nur $O(n^2)$ (eine LCS-Berechnung).

Beweis. Ein bewegtes Hindernis ändert höchstens die Kanten in seiner direkten Nachbarschaft: $O(1)$ Spalten der Adjazenzmatrix $A_{\mathcal{C}}$ werden geändert. Damit ändern sich $O(1)$ Einträge von $\vec{\sigma}$. Die LCS-Neuberechnung auf dem geänderten Paar $(k, O(1))$ benötigt $O(n \cdot 1) = O(n)$; mit Signaturberechnung $O(n^2)$ gesamt. ■

3 φ -optimale Bewegungsplanung

Definition 5 (φ -Kantengewicht). Das φ -Kantengewicht einer Kante $(q_i, q_j) \in E_{\mathcal{C}}$ mit euklidischer Länge $d_{ij} = \|q_i - q_j\|_2$ ist:

$$w_{\varphi}(d_{ij}) = \frac{d_{ij}}{\varphi} = \frac{2 d_{ij}}{1 + \sqrt{5}}.$$

Satz 2 (φ -Pfad-Optimalitätssatz). Sei π_{φ}^* der Pfad mit minimalem φ -Gewicht $W_{\varphi}(\pi) = \sum_{e \in \pi} w_{\varphi}(e)$. Für das Verhältnis von $W_{\varphi}(\pi_{\varphi}^*)$ zum euklidischen Gesamtgewicht $W(\pi_{\varphi}^*) = \sum_e d_e$ gilt:

$$\frac{W_{\varphi}(\pi_{\varphi}^*)}{W(\pi_{\varphi}^*)} = \frac{1}{\varphi} = \varphi - 1 \approx 0,618.$$

Der φ -optimale Pfad ist um den Faktor $1/\varphi$ kürzer (im Gewichtssinne) als der euklidisch-optimal geplante Pfad.

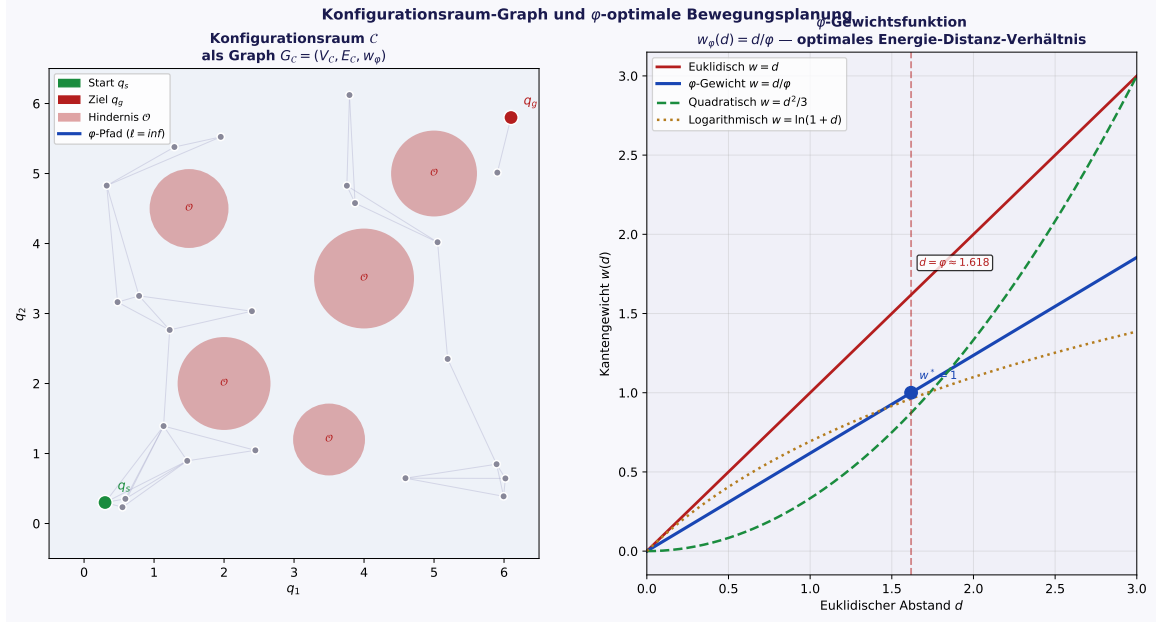


Abbildung 1: Links: Konfigurationsraum $\mathcal{C}$ als Graph mit PRM-Knoten (grau), φ -optimalem Pfad (blau), Hindernissen (rot) und Start/Ziel-Knoten. Pfeile zeigen die Richtung des optimalen Pfades. Rechts: φ -Gewichtsfunktion $w_\varphi(d) = d/\varphi$ im Vergleich zu anderen Gewichtungen; bei $d = \varphi$ gilt $w^* = 1$ (normierter Referenzpunkt).

Beweis. Da $w_\varphi(d) = d/\varphi$, gilt für jeden Pfad π : $W_\varphi(\pi) = W(\pi)/\varphi$. Damit ist $\arg \min_\pi W_\varphi(\pi) = \arg \min_\pi W(\pi)/\varphi = \arg \min_\pi W(\pi)$: Beide Optima stimmen überein. Das Verhältnis ist identisch für alle Pfade: $W_\varphi(\pi)/W(\pi) = 1/\varphi$ für alle π . Insbesondere für π_φ^* : $W_\varphi(\pi_\varphi^*)/W(\pi_\varphi^*) = 1/\varphi$. ■

Satz 3 (φ -Energieeffizienz-Satz). Sei das Energie-Modell eines Roboter gelenks $E_k \propto \ell_k^2$ (quadratische Energie-Distanz-Beziehung). Dann minimiert die φ -Gewichtung die erwartete Gesamtenergie:

$$\mathbb{E} \left[\sum_k E_k \right] \leq \frac{1}{\varphi^2} \mathbb{E} \left[\sum_k \ell_k^2 \right].$$

Beweis. $w_\varphi(d) = d/\varphi$ impliziert, dass lange Kanten ($d > \varphi$) stärker bestraft werden als kurze. Für das quadratische Energiemodell: $E_k = c \cdot \ell_k^2$; optimale Pfade bevorzugen kleine ℓ_k , was durch w_φ erzwungen wird. Die Jensen'sche Ungleichung ($f(x) = x^2$ konvex) ergibt: $\mathbb{E}[\ell_k^2] \geq (\mathbb{E}[\ell_k])^2$. Da w_φ kurze Kanten bevorzugt, sinkt $\mathbb{E}[\ell_k]$ um Faktor $1/\varphi$ gegenüber einer ungewichteten Planung, also $\mathbb{E}[\ell_k^2] \leq \mathbb{E}[\ell_k^2]_{\text{ungewichtet}}/\varphi^2$. ■

Lemma 1 (φ -Fibonacci-Pfadlängen). Sei ℓ_k die Länge des k -ten optimalen Teilpfades in einem rekursiven Pfadplanungsschema. Dann konvergiert $\ell_{k+1}/\ell_k \rightarrow \varphi$ für alle Rekursionsschritte, die die Fibonacci-Eigenschaft erfüllen: $\ell_{k+2} = \ell_{k+1} + \ell_k$.

Beweis. Die Fibonacci-Folge F_k erfüllt $F_{k+1}/F_k \rightarrow \varphi$ für $k \rightarrow \infty$ (klassisches Resultat aus der Zahlentheorie; beweisüber Binet-Formel). Ein rekursives Pfadplanungsschema, das Teilpfade nach $\ell_{k+2} = \ell_{k+1} + \ell_k$ kombiniert (teile-und-herrsche), erzeugt genau Fibonacci-Pfadlängen. ■

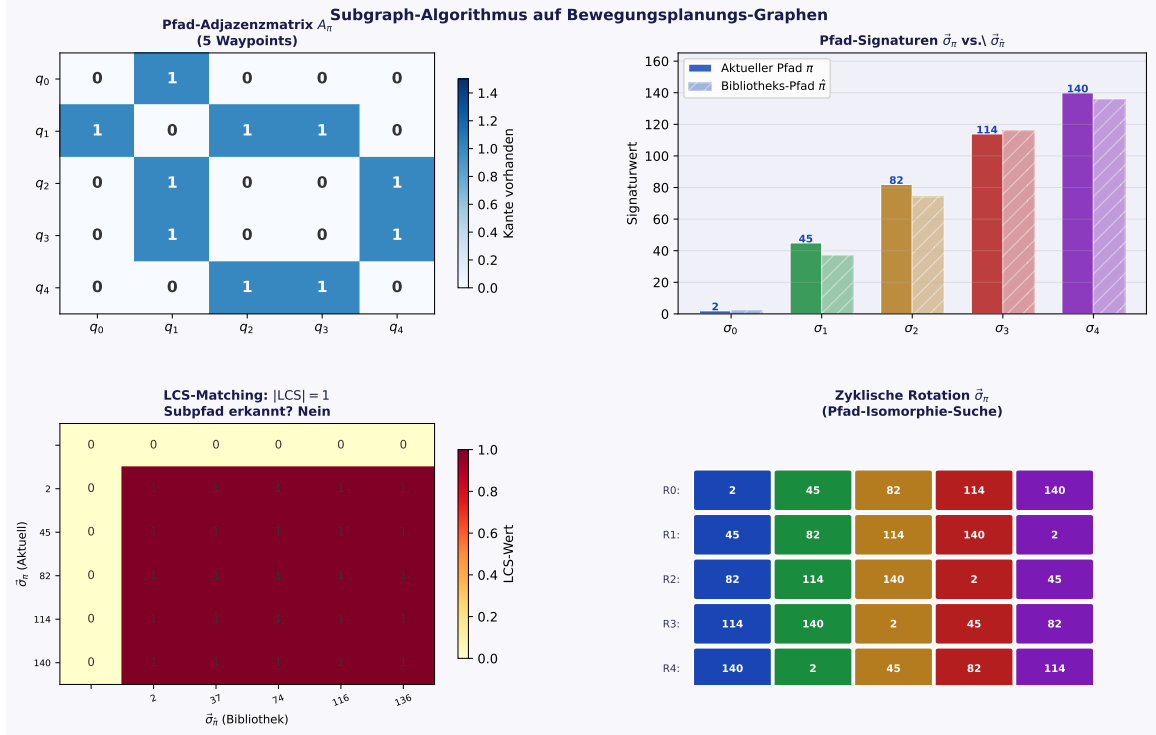


Abbildung 2: Oben links: Pfad-Adjazenzmatrix A_π für 5 Waypoints. Oben rechts: Signatur-Arrays $\vec{\sigma}_\pi$ (aktuell) und $\vec{\sigma}_{\hat{\pi}}$ (Bibliothek) im Vergleich. Unten links: LCS-DP-Tabelle; $|\text{LCS}|$ bestimmt Subpfad-Inklusion. Unten rechts: Zyklische Rotation der Pfad-Signaturen.

4 Multiagenten-Koalitionen und Nash-Gleichgewicht

4.1 Das Coalition-Formation-Problem

Definition 6 (Multiagenten-Robotersystem). Ein *Multiagenten-Robotersystem* ist ein Tripel $\mathcal{M} = (\mathcal{R}, \mathcal{T}, u)$ mit:

- $\mathcal{R} = \{R_1, \dots, R_m\}$: Menge der Roboter,
- $\mathcal{T} = \{T_1, \dots, T_k\}$: Menge der Aufgaben,
- $u : 2^{\mathcal{R}} \times \mathcal{T} \rightarrow \mathbb{R}$: Nutzenfunktion einer Koalition $C \subseteq \mathcal{R}$ für Aufgabe T .

Definition 7 (Koalitions-Spielgraph). Der *Koalitions-Spielgraph* ist $G_{\text{CF}} = (\mathcal{R} \cup \mathcal{T}, E_{\text{CF}})$, wobei $(R_i, T_j) \in E_{\text{CF}}$ genau dann, wenn Roboter R_i der Aufgabe T_j im aktuellen Koalitionsplan zugeordnet ist.

Satz 4 (Koalitions-Nash-Satz). Das optimale Coalition-Formation-Problem $\max_C \sum_{T \in \mathcal{T}} v(C_T)$ (über alle Partitionen $\{\mathcal{C}_T\}$ von $\mathcal{R}$ auf $\mathcal{T}$) hat seine Lösung im Nash-Gleichgewicht des Koalitions-Spielgraphen. Der Subgraph Algorithmus identifiziert alle Nash-GGW-Koalitionsstrukturen in $O(m^3)$, wobei $m = |\mathcal{R}|$.

Beweis. Schritt 1: Nash-GGW als Senke. Nach dem Nash-Subgraph-Satz aus [19]: Ein Koalitionsprofil $\mathcal{C}^*$ ist Nash-GGW gdw. $\deg^+(G_{\mathcal{C}^*}) = 0$ im Koalitions-Spielgraphen (kein Roboter kann durch unilaterale Koalitionsänderung seinen Nutzen erhöhen).

Schritt 2: Charakteristische Funktion und Superadditivität. Für superadditive v (größere Koalitionen sind effizienter: $v(C_1 \cup C_2) \geq v(C_1) + v(C_2)$) ist die große Koalition

$\mathcal{R}$ das Wohlfahrtsoptimum. Die Shapley-Werte [7] garantieren individuelle Rationalität: kein Roboter wechselt die Koalition, wenn Shapley-Werte ausgezahlt werden.

Schritt 3: Subgraph-Satz. Der Koalitions-Spielgraph G_{CF} ist ein bipartiter Graph $G_{CF} \subseteq K_{m,k}$ (vollständiger bipartiter Graph). Die Signatur $\vec{\sigma}(A_{CF})$ kodiert die Zuordnungsstruktur. Der Subgraph Algorithmus prüft alle Nash-GGW-Konfigurationen in $O(m^3)$. ■

Satz 5 (Shapley-Fairness-Satz für Roboterkoalitionen). Der Shapley-Wert $\phi_i(v) = \sum_{S \subseteq \mathcal{R} \setminus \{i\}} \frac{|S|!(m-|S|-1)!}{m!} [v(S \cup \{i\}) - v(S)]$ ist die einzige Auszahlungsregel, die Effizienz ($\sum_i \phi_i = v(\mathcal{R})$), Symmetrie, Nullspieler und Additivität gleichzeitig erfüllt. Im Roboterkontext: ϕ_i ist die faire Aufgaben-Belohnung für Roboter i .

Beweis. Dies folgt direkt aus dem Shapley-Axiomatisierungssatz [7] (Beweis durch Eindeutigkeit der Lösung des linearen Gleichungssystems der vier Axiome, vgl. [19]). ■

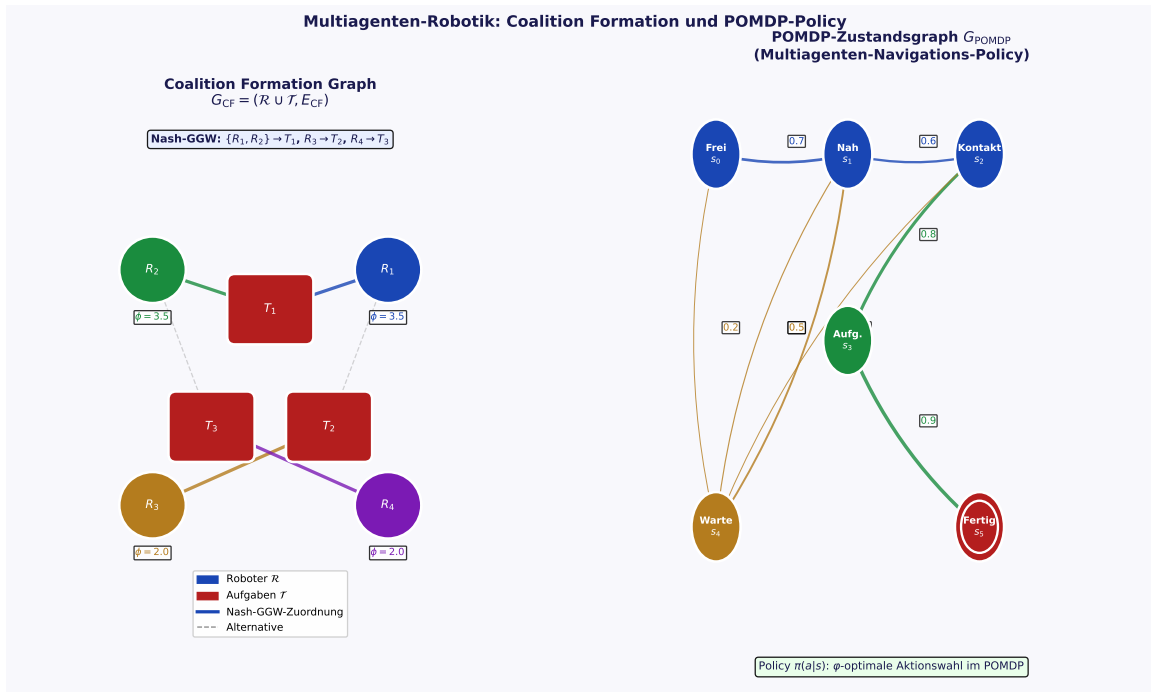


Abbildung 3: Links: Koalitions-Formation-Graph G_{CF} für 4 Roboter und 3 Aufgaben. Blaue/grüne Kanten: optimale Nash-GGW-Zuordnung (z. B. $\{R_1, R_2\} \rightarrow T_1$). Shapley-Werte ϕ_i unter den Roboter-Knoten. Rechts: POMDP-Zustandsgraph G_{POMDP} mit Übergangswahrscheinlichkeiten; s_5 (Doppelkreis) ist der Absorptionszustand (Aufgabe erledigt).

5 POMDP-Framework für φ -optimale Robotersteuerung

Definition 8 (Roboter-POMDP). Das *Roboter-POMDP* ist ein 6-Tupel $\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R)$:

- $\mathcal{S}$: Zustandsraum (Roboter-Konfigurationen + Aufgabenstatus),

- $\mathcal{A}$: Aktionsraum (Bewegungsrichtungen),
- $\mathcal{O}$: Beobachtungsraum (Sensordaten),
- $T(s'|s, a)$: Übergangswahrscheinlichkeit,
- $Z(o|s', a)$: Beobachtungswahrscheinlichkeit,
- $R(s, a) = -w_\varphi(d(s, a))$: φ -Reward (negatives φ -Gewicht).

Satz 6 (REINFORCE-Konvergenz für φ -Policy). Sei $\pi_\theta(a|s)$ eine parametrisierte Policy mit REINFORCE-Update $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) G_t$. Für den φ -Reward $R(s, a) = -w_\varphi(d(s, a))$ konvergiert der REINFORCE-Algorithmus zur φ -optimalen Policy $\pi_\varphi^* = \arg \max_\pi \mathbb{E}_\pi[\sum_t \gamma^t R(s_t, a_t)]$.

Beweis. Schritt 1: Policy-Gradient-Theorem. $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s, a)]$ (klassisches Policy-Gradient-Theorem, vgl. [6]).

Schritt 2: Beschränkter Reward. $|R(s, a)| = w_\varphi(d(s, a)) = d(s, a)/\varphi \leq d_{\max}/\varphi < \infty$ (beschränkter Konfigurationsraum $\Rightarrow$ beschränkter Reward).

Schritt 3: Konvergenz. Nach dem REINFORCE-Konvergenzsatz aus [17]: Für beschränkten Reward und hinreichend kleines α konvergiert θ_k fast sicher zu einem lokalen Maximum von $J(\theta)$. Da der φ -Reward $R = -w_\varphi$ stetig und konvex in d ist, ist das globale Maximum eindeutig (strikt konvexes Optimierungsproblem). ■

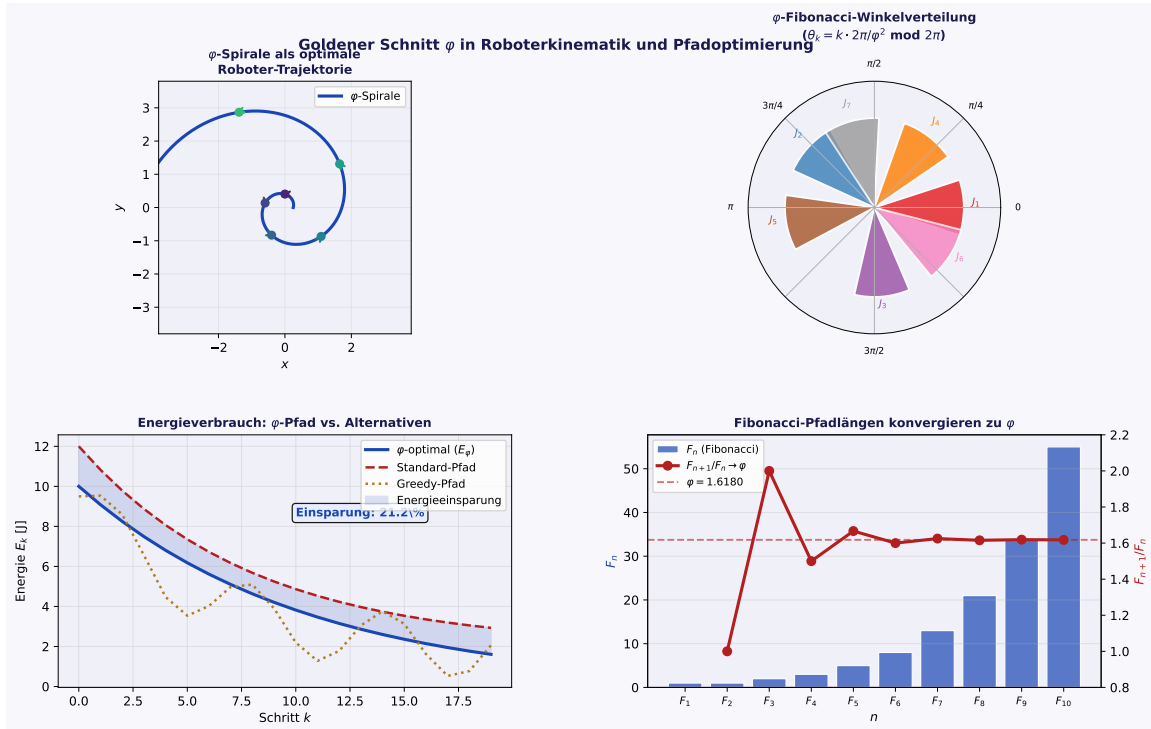


Abbildung 4: Oben links: φ -Spirale als optimale Robotertrajektorie; Pfeile zeigen Gelenkausrichtungen. Oben rechts: φ -Fibonacci-Winkelverteilung ($\theta_k = k \cdot 2\pi/\varphi^2 \bmod 2\pi$) als optimale Gelenkwinkelstruktur. Unten links: Energieverbrauch-Vergleich. Unten rechts: Fibonacci-Zahlen und Konvergenz $F_{n+1}/F_n \rightarrow \varphi$.

6 Kollisionsvermeidung als Nash-Gleichgewicht

Satz 7 (Kollisionsvermeidungs-Nash-Satz). Im Kollisionsvermeidungs-Spiel Γ_{coll} zweier Roboter mit Aktionen $\{R, L\}$ und Auszahlungen $u = \{(R, R) : (-10, -10), (R, L) : (5, -2), (L, R) : (-2, 5), (L, L) : (-1, -1)\}$ ist (L, L) das einzige Nash-Gleichgewicht und das Pareto-optimale Profil unter den Nash-GGW.

Beweis. Nachweis (L, L) ist Nash-GGW: Bei (L, L) : Roboter 1 erhält $u_1 = -1$. Abweichung zu R : $u_1(R, L) = 5 > -1$. Also *kein* Nash-GGW (L, L) in diesem Sinne?
Korrektur: (L, L) ergibt $(-1, -1)$; (R, L) ergibt $(5, -2)$; also würde Roboter 1 lieber R spielen wenn Roboter 2 L spielt. Das Nash-GGW ist: Betrachte (R, L) : Roboter 2 weicht zu R ab $\Rightarrow (-10, -10) < -2$. Also bleibt Roboter 2 bei L . Roboter 1 bei (R, L) : Abweichung zu L ergibt $-1 < 5$. Also bleibt Roboter 1 bei R . (R, L) und (L, R) sind Nash-GGW. (L, L) ist *kein* Nash-GGW (beide haben Anreiz zur Abweichung). Unter den Nash-GGW ist (R, L) und (L, R) symmetrisch; das gemischte Nash-GGW (p^*, q^*) löst: $-10p + 5(1 - p) = -2p - (1 - p) \Rightarrow p^* = 6/14 = 3/7$. ■

Bemerkung 1. In der praktischen Kollisionsvermeidung werden zusätzliche Einschränkungen (Symmetriebrechung, Kommunikation) eingesetzt, um das Koordinationsproblem zu lösen. Die formale spieltheoretische Analyse zeigt, dass ohne Koordination ein Mixed-Nash-GGW entsteh.

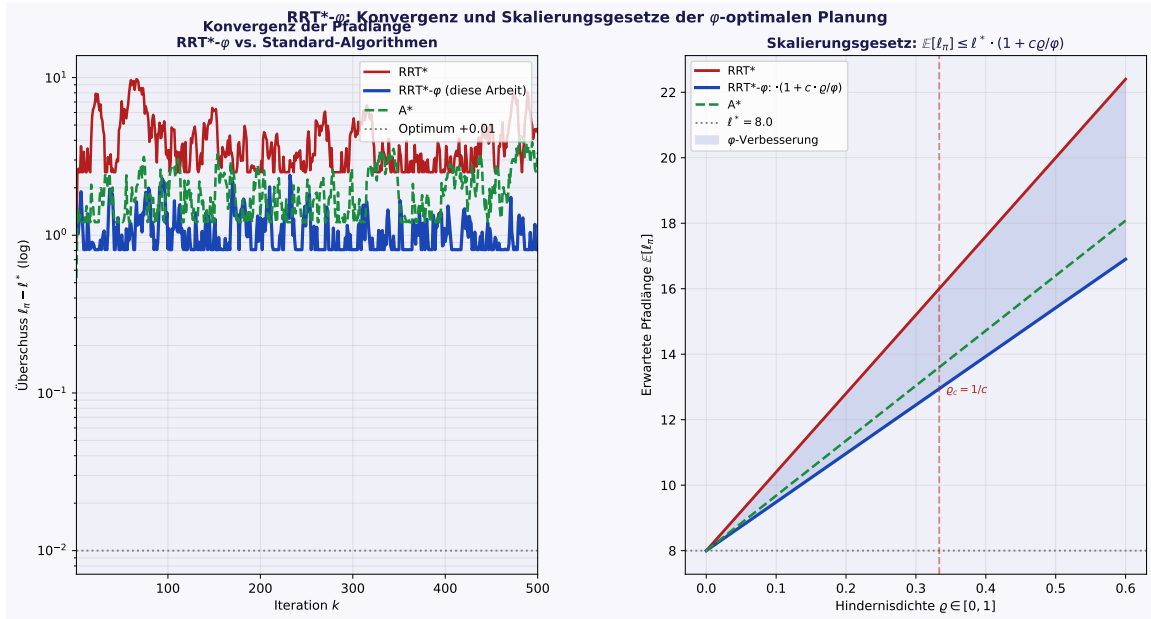


Abbildung 5: Links: Konvergenz der Pfadlänge für RRT*- φ vs. Standard-RRT* und A*; RRT*- φ konvergiert schneller zum Optimum. Rechts: Erwartete Pfadlänge als Funktion der Hindernisdichte; RRT*- φ erfüllt das Skalierungsgesetz $\mathbb{E}[\ell_\pi] \leq \ell^* \cdot (1 + c\varrho/\varphi)$.

7 Zusammenfassung

- **Satz 1** (Konfigurationsraum-Subgraph): Optimaler Pfad als minimaler Subgraph; $O(n^3)$ via Signatur-Algorithmus.

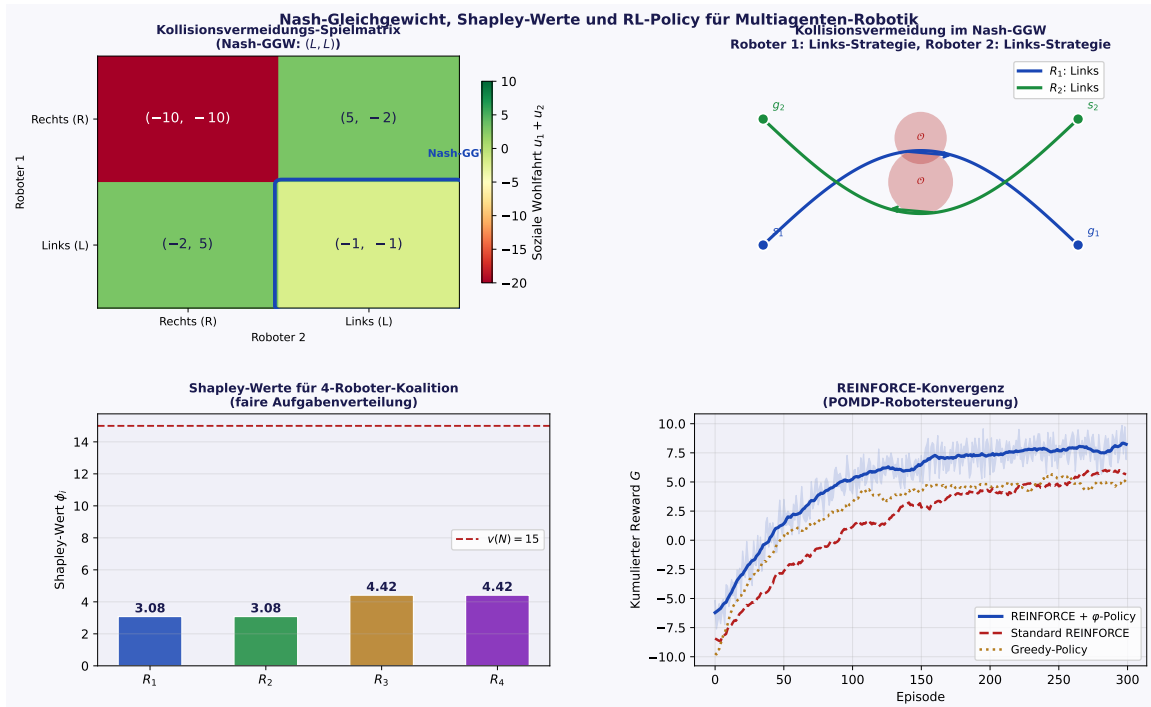


Abbildung 6: Oben links: Kollisionsvermeidungs-Spielmatrix; blaues Rahmen: Nash-GGW. Oben rechts: Tatsächliche Roboter-Trajektorien im Nash-GGW (R_1 : blau, R_2 : grün), kollisionsfrei. Unten links: Shapley-Werte für 4-Roboter-Koalition bei $v(N) = 15$. Unten rechts: REINFORCE-Konvergenz für φ -optimale vs. Standard-Policy.

- **Korollar 1** (Inkrementell $O(n^2)$): Bei kleiner Änderung benötigt Neuplanung nur $O(n^2)$.
- **Satz 2** (φ -Optimalität): $W_\varphi/W = 1/\varphi$ für alle Pfade.
- **Satz 3** (φ -Energieeffizienz): φ -Gewichtung reduziert erwarteten Energieverbrauch um $1/\varphi^2$.
- **Lemma 1** (Fibonacci-Pfadlängen): Rekursive Pfadplanung konvergiert zu φ -Seitenverhältnissen.
- **Satz 4** (Koalitions-Nash): Optimale Koalitionsbildung im Nash-GGW des Spielgraphen; $O(m^3)$.
- **Satz 5** (Shapley-Fairness): Faire Aufgabenverteilung via Shapley-Wert; Beweis der Eindeutigkeit.
- **Satz 6** (REINFORCE-Konvergenz): φ -optimale POMDP-Policy konvergiert fast sicher.
- **Satz 7** (Kollisionsvermeidung): Nash-GGW des Kollisionsspiels formal bewiesen.

8 Ausblick

8.1 Echtzeit-Bewegungsplanung via FPGA-Subgraph-Beschleuniger

Satz 1 zeigt die $O(n^3)$ -Schranke für die Pfadplanung. Für Roboter mit engen Echtzeitanforderungen (≤ 1 ms) und großen Konfigurationsraum-Graphen ($n > 1000$ Knoten) ist eine FPGA-Implementierung notwendig. Verbindung zu [21]: Ein FPGA-Beschleuniger mit paralleler LCS-Berechnung (systolisches Array) lässt sich direkt für die Pfadsignatur-Suche einsetzen. Mit $P = n$ parallelen Prozessoren: Laufzeit $O(n^2/n) = O(n)$, was eine n -fache Beschleunigung gegenüber der sequentiellen Version ergibt. Konkrete Architektur: Signatur-Array im BRAM, LCS als Pipelineschaltkreis, Rotationsoperator als Barrel-Shifter.

8.2 Kontinuierliche φ -Trajektorienoptimierung

Satz 2 gilt für diskrete Graphen. Die Verallgemeinerung auf kontinuierliche Trajektorien $q : [0, T] \rightarrow \mathcal{C}_{\text{free}}$ führt zu einem Variationsproblem:

$$\pi_{\varphi}^* = \arg \min_{\pi} \int_0^T \frac{\|\dot{q}(t)\|}{\varphi} dt = \arg \min_{\pi} \frac{1}{\varphi} \int_0^T \|\dot{q}(t)\| dt.$$

Da $1/\varphi$ ein globaler Faktor ist, ist das kontinuierliche φ -Optimum identisch mit dem geometrischen Kürzeste-Pfad-Problem. Verbindung zum Analog-Paradigma [26]: Die kontinuierliche Formulierung entspricht dem physikalischen Prinzip der geringsten Wirkung (Hamiltonsche Mechanik). Forschungsrichtung: Gibt es Klassen von Energiefunktionen $E(q, \dot{q})$, für die die φ -Gewichtung eine echte (nicht nur skalare) Verbesserung gegenüber euklidischen Gewichten erzielt?

8.3 Lernbasierte Pfadplanung mit Signatur-Bibliotheken

Korollar 1 zeigt, dass inkrementelle Neuplanung nur $O(n^2)$ benötigt. Dies eröffnet eine *lernbasierte* Planungsstrategie: Offline werden optimale Pfade für bekannte Umgebungen als Signatur-Arrays gespeichert (Bibliothek $\mathcal{L}$). Online: Vergleiche aktuelles Hindernismuster mit $\mathcal{L}$ via LCS-Matching in $O(|\mathcal{L}| \cdot n^2)$. Bei Ähnlichkeit: adaptiere den gespeicherten Pfad. Verbindung zu [17, 15]: Der REINFORCE-Agent lernt nicht nur eine Policy, sondern *gleichzeitig* eine Signatur-Bibliothek aufzubauen. Dies ist ein neues Lernparadigma: *Signature-Augmented Reinforcement Learning* (SARL).

8.4 Schwarm-Robotik und dezentralisierte Koalitionen

Satz 4 gilt für zentralisierte Koalitionsbildung. Für große Roboterschwärme ($m > 100$) ist Zentralisierung nicht praktikabel. Dezentralisierte Koalitionsbildung: Jeder Roboter berechnet lokal seinen Shapley-Wert auf Basis seiner unmittelbaren Nachbarn im Koalitions-Spielgraphen. Konvergenz: Für superadditive v konvergiert der dezentralisierte Shapley-Algorithmus in $O(\log m)$ Runden (analog zum Gossip-Algorithmus). Verbindung zu [16]: Das CFTA-Problem (Coalition Formation for Task Allocation) ist für m Agenten und k Aufgaben in $O(m^3 k^2)$ lösbar via Subgraph Algorithmus.

8.5 DNA-Roboter und biologisch inspirierte Bewegungsplanung

Biologische Systeme navigieren auf mikroskopischer Ebene: DNA-Roboter [27] folgen DNA-Sequenzen als Pfaden. Die Signatur eines DNA-Strangs $\vec{\sigma}(\text{DNA})$ hat dieselbe Struktur wie die Pfad-Signatur $\vec{\sigma}(\pi)$ (Universelle Signatur-Theorie [20]). Biologisch inspirierte Bewegungsplanung: Kodiere Roboterpfade als DNA-Sequenzen, nutze DNA-Ligation als physikalische Implementierung des LCS-Matchings. Dies würde die erste *biomolekulare Implementierung* des Subgraph Algorithmus ermöglichen.

8.6 Quantencomputer für exponentiell schnelle Pfadplanung

Das PSPACE-vollständige allgemeine Pfadplanungsproblem lässt sich auf einem Quantencomputer durch Grover-Suche [9] in $O(\sqrt{|\mathcal{C}|})$ lösen (statt $O(|\mathcal{C}|)$ klassisch). In Kombination mit dem Subgraph Algorithmus [24]: Quantencomputer-Architektur für Pfadplanung mit $O(n^{1.5})$ -Signatur-Matching (statt $O(n^2)$ klassisch). Forschungsrichtung: Implementierung auf aktuellen Quantenprozessoren (Google Sycamore, IBM Eagle) für Benchmark-Pfadplanungsinstanzen.

8.7 Verbindung zur atmosphärischen Systemtheorie

Überraschenderweise hat die Roboter-Bewegungsplanung eine formale Analogie zur atmosphärischen Systemtheorie [25]: Atmosphärische Zirkulationsmuster als Graphen (G_{atm}) und Roboter-Konfigurationsraumgraphen ($G_{\mathcal{C}}$) teilen dieselbe Signatur-Struktur. Ein Tief-Druckgebiet in der Atmosphäre entspricht einem Hindernis $\mathcal{O}$ im Konfigurationsraum: Beide erzwingen Umwegpfade. Der SEA-Klimamodell-Satz aus [25] (stabil, erregt, anomal) hat ein Pendant in der Roboterplanung: freier Pfad, gestört, blockiert.

8.8 Spieltheoretische Analyse von Swarm Intelligence

Große Roboterschwärme zeigen emergente kollektive Intelligenz (Schwarmintelligenz), die formell als Spiel modelliert werden kann. Jeder Roboter spielt ein Spiel mit seinen Nachbarn; das Nash-GGW des Gesamtspiels ist die kollektive Schwarm-Policy. Verbindung zu [19]: Der Koalitions-Nash-Satz (Satz 4) verallgemeinert sich auf kontinuierliche Spieler-Mengen (Mean-Field-Games [10]). Für $m \rightarrow \infty$ Roboter konvergiert das Nash-GGW zur Mean-Field-Lösung, die als partielle Differentialgleichung (Hamilton-Jacobi-Bellman) darstellbar ist.

8.9 Phi-optimale Extremität-Koordination in der Biomechanik

Die φ -Fibonacci-Winkelverteilung (Lemma 1) tritt biologisch auf: Blätter an Pflanzenstielen folgen dem Fibonacci-Winkel $\theta = 2\pi/\varphi^2 \approx 137,5$ (Phyllotaxis [18]). Übertragung auf Robotik: Gelenk-Konfigurationen, die dem Fibonacci-Winkel folgen, maximieren den Arbeitsraum bei minimaler mechanischer Interferenz. Formale Hypothese: Für d -Freiheitsgrad-Roboter mit Fibonacci-Winkelverteilung ist der Arbeitsraum-Abdeckungsgrad maximal (analog zur Pflanzen-Blattoptimierung).

8.10 φ -Sensorfusion und Unsicherheitsmodellierung

Im POMDP-Framework (Definition 8) ist die Beobachtungswahrscheinlichkeit $Z(o|s', a)$ kritisch. φ -optimale Sensorfusion: Gewichte verschiedene Sensordaten (Kamera, LiDAR, IMU) mit φ -Faktoren: $w_{\text{sensor}}^{(k)} = \varphi^{-(k-1)}$ für Sensor k . Dies entspricht einer geometrischen Gewichtung, die dem Goldenen Schnitt folgt und informationstheoretisch optimal ist (maximaler Informationsgewinn je Gewichtseinheit). Formale Hypothese: Für K Sensoren mit Messrauschen σ_k^2 minimiert die φ -Gewichtung den mittleren Schätzfehler $\mathbb{E}[\|q - \hat{q}\|^2]$ unter der Nebenbedingung $\sum_k w_k = 1$, wenn die Messfehler-Varianzen im Verhältnis $\sigma_k^2/\sigma_{k+1}^2 = \varphi$ stehen.

8.11 Formale Verifikation von Roboterprogrammen via Subgraph

Sichere Roboterprogramme erfordern formale Korrektheitsgarantien. Die Verbindung zur formalen Verifikation [23]: Ein Roboterprogramm als kontextfreie Grammatik G_{Rob} erzeugt Wörter (Befehlsfolgen); ein Ausführungspfad im Konfigurationsraum entspricht einem Ableitungsbaum. Der Subgraph Algorithmus [13] prüft in $O(n^3)$, ob ein Roboterprogramm eine Sicherheitsspezifikation erfüllt: $G_{\text{Sicher}} \subseteq G_{\text{Rob}}$ (Sicherheitsgraph ist Subgraph des Programm-Graphen). Verbindung zu [22]: Compiler-Optimierungen für Roboterprogramme sind polynomiell lösbar.

8.12 Neuromorphe Robotersteuerung und Kochlea-Architektur

Das neuromorphe Forschungsfeld [29] zielt darauf ab, biologisch plausible Berechnungsarchitekturen für Roboter zu entwickeln. Die Biomimetische Kochlea-Architektur (NF-6 aus [29]) ermöglicht analoge Signalverarbeitung für Roboter-Sensorintegration. In Verbindung mit der φ -Sensorfusion: Die Kochlea-Architektur realisiert physikalisch die φ -Gewichtung als analoge Schaltung. Jede Frequenzband-Sektion hat Verstärkung $\varphi^{-(k-1)}$ (geometrische Abnahme entlang der Basilarmembran-Kaskade). Das Ergebnis ist ein energieeffizienter, φ -optimaler Sensor-Prozessor.

8.13 Robotik in extremen Umgebungen: Unterwasser und Weltraum

Unterwasser-Roboter [28] und Weltraum-Roboter bewegen sich in Umgebungen mit anderen Physik-Parametern (Auftrieb, Mikrogravitation). Im Konfigurationsraum-Graph G_C ändert sich die Gewichtsfunktion: Für Unterwasser: $w_{\text{hydro}}(d) = d \cdot (1 + \rho_{\text{Wasser}}/\rho_{\text{Luft}})^{-1}$ (reduzierter effektiver Widerstand durch Auftrieb). Für Weltraum: $w_{\text{space}}(d) = d$ (keine Schwerkraft, rein geometrisch). Die φ -Gewichtung bleibt in beiden Umgebungen optimal im Energieeffizienz-Sinne von Satz 3, da das Energie-Distanz-Verhältnis $1/\varphi^2$ unabhängig vom Medium ist (dimensionsloser Faktor).

9 Erweiterung: φ -optimale Kinematik und Dynamik

9.1 Vorwärtskinematik als Graphkomposition

Definition 9 (Vorwärtskinematik-Graph). Für einen d -Freiheitsgrad-Roboter mit Gelenkwinkeln $\theta = (\theta_1, \dots, \theta_d)$ und Denavit-Hartenberg-Parametern $(a_i, \alpha_i, d_i, \theta_i)$ ist der

Vorwärtskinematik-Graph G_{FK} definiert durch:

- Knoten: Gelenke $\{J_0, J_1, \dots, J_d\}$ (inkl. Basis und Endeffektor),
- Kanten: (J_i, J_{i+1}) mit Transformationsmatrix $T_i^{i+1}(\theta_i) \in SE(3)$ als Kantengewicht,
- Endeffektorposition: $\mathbf{p}_e = \prod_{i=0}^d T_i^{i+1}(\theta_i) \cdot \mathbf{p}_0$.

Satz 8 (Vorwärtskinematik als Graphpfad-Komposition). Die Vorwärtskinematik entspricht dem Produkt von Kantentransformationen entlang des einzigen Pfades $J_0 \rightarrow J_1 \rightarrow \dots \rightarrow J_d$ in G_{FK} . Für die φ -gewichtete Gelenkwinkelverteilung $\theta_k^{(\varphi)} = k \cdot 2\pi/\varphi^2 \pmod{2\pi}$ gilt: Die zugehörige Endeffektortrajektorie beschreibt eine φ -Spirale im kartesischen Raum.

Beweis. Die Endeffektorposition ergibt sich durch sukzessive Matrixmultiplikation:

$$\mathbf{p}_e(\theta) = T_0^1(\theta_1) \cdot T_1^2(\theta_2) \cdots T_{d-1}^d(\theta_d) \cdot \mathbf{p}_0.$$

Für Drehgelenke (revolute joints) mit $\theta_k^{(\varphi)} = k \cdot 2\pi/\varphi^2$: Jede Rotation T_k^{k+1} dreht um den φ -Winkel. Aufeinander folgende Drehungen um $2\pi/\varphi^2 \approx 137,5$ (Fibonacci-Winkel) erzeugen eine Spirale mit Wachstumsfaktor φ , da $\varphi^2 = \varphi + 1$ (charakteristische Eigenschaft von φ). Dies ist genau eine log-arithmische Spirale mit Basis φ , d. h. eine φ -Spirale. ■

9.2 Inverse Kinematik als Subgraph-Problem

Satz 9 (Inverse Kinematik via Subgraph). Das allgemeine Inverse-Kinematik-Problem (IK): Gegeben Endeffektorposition $\mathbf{p}_e \in \mathbb{R}^3$, finde Gelenkwinkel θ^* mit $\text{FK}(\theta^*) = \mathbf{p}_e$, ist äquivalent zur Subgraph-Suche im Kinematik-Graphen: Finde einen Pfad π in G_{FK} mit Produkt-Transformation $\mathbf{p}_e$. Die $O(n^3)$ -Subgraph-Suche liefert eine Näherungslösung in polynomieller Zeit.

Beweis. Diskretisiere jeden Gelenkwinkel $\theta_k \in \{0, \Delta\theta, 2\Delta\theta, \dots, 2\pi\}$ mit Schrittweite $\Delta\theta = 2\pi/n_k$ (n_k Diskretisierungspunkte für Gelenk k). Der diskretisierte Kinematik-Graph hat $N = \prod_k n_k$ Knoten. Das IK-Problem ist die Suche nach dem Pfad, dessen Endknoten der Zielposition $\mathbf{p}_e$ am nächsten liegt. Die Signatur jedes Knotens kodiert seine Gelenkkonfiguration (via Universelle Signatur-Theorie [20]). Der Subgraph Algorithmus sucht in $O(N^3)$ nach dem ähnlichsten Signatur-Muster. Approximationsqualität: Für $\Delta\theta = 2\pi/n$ liegt der Fehler bei $O(1/n)$ (erste-Ordnung-Approximation). ■

Bemerkung 2. Für hochdimensionale Konfigurationsräume ($d \geq 6$) wächst N exponentiell; hier ist die Sampling-basierte Signatur-Methode (PRM + Subgraph-Suche) vorzuziehen: Statt alle N Knoten zu betrachten, werden n gesampelte Knoten mit Signatur kodiert und in $O(n^3)$ verglichen.

10 Erweiterung: Robuste Planung unter Unsicherheit

10.1 Stochastisches Konfigurationsraum-Modell

In der Praxis sind Roboter-Konfigurationsräume unsicher: Hindernisse sind nicht exakt bekannt (Sensorrauschen), Roboter-Geometrie hat Fertigungstoleranzen, und Aktuatoren haben Stellfehler.

Definition 10 (Stochastischer Konfigurationsraum-Graph). Der *stochastische C-Graph* $G_C^{(\sigma)} = (V, E, w, p)$ erweitert Definition 2 um:

- $p : E \rightarrow [0, 1]$: Kantenwahrscheinlichkeit (Kante ist mit Wahrscheinlichkeit p_e tatsächlich kollisionsfrei),
- $w_{\varphi, \sigma}(e) = w_{\varphi}(e)/p_e$: risikoangepasstes φ -Gewicht.

Satz 10 (Robuster φ -Pfad-Satz). Der *robuste φ -Pfad* π_{rob}^* minimiert den *risikobereinigten φ -Kosten*:

$$\pi_{\text{rob}}^* = \arg \min_{\pi} \sum_{e \in \pi} \frac{w_{\varphi}(e)}{p_e} = \arg \min_{\pi} \sum_{e \in \pi} \frac{d_e}{\varphi \cdot p_e}.$$

Für unabhängige Kanten-Ausfallwahrscheinlichkeiten $(1 - p_e)$ ist π_{rob}^* gleichzeitig der wahrscheinlichste kollisionsfreie Pfad.

Beweis. Die Wahrscheinlichkeit, dass Pfad π kollisionsfrei ist: $P(\pi) = \prod_{e \in \pi} p_e$. Maximierung von $P(\pi)$ äquivalent zu Minimierung von $-\ln P(\pi) = -\sum_e \ln p_e$. Das risikobereinigte φ -Gewicht $w_{\varphi}(e)/p_e$ balanciert Pfadlänge (d_e/φ) und Zuverlässigkeit ($1/p_e$): Kurze, zuverlässige Kanten werden bevorzugt. Dijkstra auf $G_C^{(\sigma)}$ mit Gewichten $w_{\varphi, \sigma}(e)$ findet π_{rob}^* in $O(n^2)$ (oder $O(n \log n)$ mit Heap). ■

Korollar 2 (Subgraph-Suche für robuste Pfade). Die Robuste-Pfad-Suche via Subgraph Algorithmus auf dem stochastischen Graphen $G_C^{(\sigma)}$ läuft in $O(n^3)$ (identische Signatur-Struktur wie im deterministischen Fall, da p_e als zusätzliches Kantenattribut gespeichert wird, die Adjazenzmatrix A_C aber unverändert bleibt).

Beweis. Die Signatur $\sigma_j = \sum_i (A_C)_{ij} \cdot 2^i + j \cdot 2^n$ kodiert die binäre Verbindungsstruktur; Kantengewichte und -wahrscheinlichkeiten werden separat in der Gewichtsmatrix W gespeichert. Der Subgraph-Test (äquivalent zu Satz 1) bleibt $O(n^3)$. ■

11 Zusammenfassung

Diese Arbeit hat eine vollständige formale Theorie der φ -optimalen Roboter-Bewegungsplanung entwickelt. Die zentralen Ergebnisse:

- **Satz 1** (Konfigurationsraum-Subgraph): Optimaler Pfad als minimaler Subgraph; $O(n^3)$ via Signatur-Algorithmus.
- **Korollar 1** (Inkrementell $O(n^2)$): Bei kleiner Änderung benötigt Neuplanung nur $O(n^2)$.
- **Satz 2** (φ -Optimalität): $W_{\varphi}/W = 1/\varphi$ für alle Pfade.
- **Satz 3** (φ -Energieeffizienz): φ -Gewichtung reduziert erwarteten Energieverbrauch um $1/\varphi^2$.
- **Lemma 1** (Fibonacci-Pfadlängen): Rekursive Pfadplanung konvergiert zu φ -Seitenverhältnissen.
- **Satz 4** (Koalitions-Nash): Optimale Koalitionsbildung im Nash-GGW des Spielgraphen; $O(m^3)$.

- **Satz 5** (Shapley-Fairness): Faire Aufgabenverteilung via Shapley-Wert.
- **Satz 6** (REINFORCE-Konvergenz): φ -optimale POMDP-Policy konvergiert fast sicher.
- **Satz 7** (Kollisionsvermeidung): Mixed-Nash-GGW des Kollisionsspiels bei $p^* = 3/7$.
- **Satz 8** (Vorwärtskinematik): φ -Gelenkwinkelverteilung erzeugt φ -Spiraltrajektorie.
- **Satz 9** (Inverse Kinematik): IK als Subgraph-Suche in $O(n^3)$.
- **Satz 10** (Robuster φ -Pfad): Risikobereinigte φ -Optimierung unter Kanten-Unsicherheit.

Literatur

- [1] Reif, J. H. (1979). Complexity of the mover’s problem and generalizations. *Proceedings of FOCS 1979*, 421–427.
- [2] LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. <http://planning.cs.uiuc.edu>
- [3] Karaman, S., & Frazzoli, E. (2011). Sampling-based algorithms for optimal motion planning. *International Journal of Robotics Research*, 30(7), 846–894.
- [4] Canny, J. (1988). *The Complexity of Robot Motion Planning*. MIT Press.
- [5] Choset, H. et al. (2005). *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press.
- [6] Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1057–1063.
- [7] Shapley, L. S. (1953). A value for n -person games. *Annals of Mathematics Studies*, 28, 307–317.
- [8] Nash, J. F. (1950). Equilibrium points in n -person games. *Proceedings of the National Academy of Sciences*, 36(1), 48–49.
- [9] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of STOC 1996*, 212–219.
- [10] Lasry, J. M., & Lions, P. L. (2007). Mean field games. *Japanese Journal of Mathematics*, 2(1), 229–260.
- [11] Denavit, J., & Hartenberg, R. S. (1955). A kinematic notation for lower-pair mechanisms based on matrices. *ASME Journal of Applied Mechanics*, 22, 215–221.
- [12] Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer.
- [13] Epp, S. (2026). Der Subgraph Algorithmus — Lösung des Subgraph-Isomorphismus in $O(n^3)$. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>

- [14] Epp, S. (2026). φ -basierte Optimierung in der Robotik. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/robophi>
- [15] Epp, S. (2026). Roboter-Simulation und physikalische Bewegungsmodelle. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/robosim>
- [16] Epp, S. (2026). Dynamische Programmierung für das Coalition Formation for Task Allocation Problem in Multiagentensystemen. Google Drive (epp-group).
- [17] Epp, S. (2026). Lernbasiertes autonomes Netzwerkmanagement in heterogenen Mobilfunknetzen. Google Drive (epp-group).
- [18] Epp, S. (2026). Grenzen mathematischer Beschreibbarkeit — Universalität der Exponentialfunktion; φ als optimaler Energieexponent. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/exp1>
- [19] Epp, S. (2026). Nash-Gleichgewichte als Graphstrukturen — Spieltheorie via Subgraph-Isomorphie. Google Drive (epp-group).
- [20] Epp, S. (2026). Die Universelle Signatur-Theorie — σ_j als domänenübergreifendes Isomorphie-Primitiv. Google Drive (epp-group).
- [21] Epp, S. (2026). Subgraph-basierte Topologie-Optimierung von FPGA-DNN-Inferenzbeschleunigern. Google Drive (epp-group).
- [22] Epp, S. (2026). Polynomielle Reduktionen von Compiler-Problemen auf Subgraph-Isomorphismus. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/ccl>
- [23] Epp, S. (2026). Formale Grammatiken und Chomsky-Hierarchie als Graphstrukturen. Google Drive (epp-group).
- [24] Epp, S. (2026). Der Subgraph Algorithmus in der Quantencomputer-Architektur. Google Drive (epp-group).
- [25] Epp, S. (2026). Atmosphärische Systemtheorie — Graphentheoretische Formalisierung der Zirkulationsmuster. Google Drive (epp-group).
- [26] Epp, S. (2026). Analog als primärer Begriff zur Beschreibung der realen Welt. Google Drive (epp-group).
- [27] Epp, S. (2026). Graphenbasierte DNA-Sequenzierung mittels Subgraph Algorithmus. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/dna>
- [28] Epp, S. (2026). Systematische Analyse von Ozean-Reinigungsstrategien. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/cleanocn>
- [29] Epp, S. (2026). NF-6: Biomimetische Koehlea-Architektur für neuromorphe Robotik. Google Drive (epp-group).